HACKFRONT INDIA • EDITION 01 • THEME T4: FINTECH

DEBITLENS

Find it. Understand it. Stop it.

A team discussion and product-planning brief for PS T4.1 — “The Auto Debits Nobody Can Find”

01	02	03
DISCOVER	**EXPLAIN**	**CANCEL**
Find recurring commitments	Resolve merchant identity	Guide the correct stop process

PURPOSE

Use this document to align the team on the problem, understand existing products, decide where to differentiate, and agree on a realistic prototype before implementation begins.

Prepared for internal team discussion | 30 October 2026

Working concept name only — product name, market claims and technical scope remain open for team validation.

1. Executive summary

The recommendation

Build DebitLens as an evidence-backed auto-debit intelligence and cancellation assistant. It should combine recurring-payment discovery, merchant explanation, change detection and payment-mechanism-aware cancellation guidance. Do not position it as the first subscription tracker; several products already cover substantial parts of that space.

The problem in one sentence

People cannot easily see every recurring financial commitment across statements, payment notifications and payment rails; confusing descriptors hide the merchant, irregular renewals escape simple tracking, and users may not know which service and payment authorisation must each be cancelled.

What we propose

DebitLens takes synthetic bank-statement rows and payment notifications, normalises them, identifies probable recurring patterns, resolves merchant descriptors using a curated reference set, explains the evidence and provides a separate checklist for cancelling the underlying service and the relevant payment authorisation where applicable.

Why this direction is worth exploring

- The hackathon asks for more than a recurring-payment list: irregular and annual payments, merchant interpretation, price increases/trial conversion/continued charges, correct cancellation paths, and annual cost visibility [1].
- Existing products already offer subscription calendars, SMS/email capture, statement imports, price-change alerts, renewal reminders and cancellation guides. These are not unique features by themselves [2–7].
- NPCI has already moved toward easier UPI mandate discovery and lifecycle management. DebitLens should therefore focus on a broader commitment view and on connecting payment evidence to the underlying merchant/service, rather than claiming that UPI mandates cannot be viewed together [8–10].

The decision we need from the team

Agree on one narrow end-to-end prototype: import a prepared synthetic dataset → identify a set of recurring commitments → explain at least one ambiguous merchant → flag at least one non-monthly or changing commitment → show a correct cancellation playbook → record the user's completion status.

A realistic positioning statement

"DebitLens helps consumers understand recurring payments across their financial records, see the evidence behind each identified commitment, and follow the correct steps to stop the service and/or its payment authorisation."

2. What HackFront is asking us to build

Source basis: HackFront India Edition 01 Problem Statements, Fintech theme T4, PS T4.1, pages 11–12. The points below are a structured interpretation of the supplied problem statement, not additional requirements invented by the team [1].

Hackathon requirement	What it means in the product	Evidence the demo should show
Discover recurring commitments	Find recurring payments from transaction history and payment notifications, including non-monthly and irregular intervals.	A labelled input dataset and a detected commitment list with frequency and supporting transactions.
Interpret merchant strings	Convert opaque or shortened descriptors into a recognisable merchant/service, while exposing uncertainty.	At least one descriptor resolved to a merchant, with the rule or reference evidence displayed.
Detect important changes	Identify price rises, free-trial-to-paid conversion, and charges continuing after service use has stopped.	A working price-change case, trial conversion case and a user-confirmed low/unused-use case.
Give the correct cancellation path	Show the steps for the payment mechanism and explain what happens if the merchant does not respond.	A real-service cancellation procedure tested end to end where feasible; distinguish service cancellation from mandate revocation.
Show annual cost	Calculate the annual financial commitment rather than only a monthly view.	Annual total and a transparent explanation of how monthly/quarterly/annual estimates are calculated.

Important constraints stated in the brief

- Synthetic statements and notifications should be generated for the prototype. The brief says not to touch real financial data [1].
- The merchant-string reference set can be assembled manually. The brief explicitly suggests that careful reference data may outperform a model on the cases that matter [1].
- The “layer beneath” is discovery of irregular commitments: annual renewal, slightly changing amounts, and charges that resume after a pause [1].
- The judges reward measured detection rate, merchant interpretation accuracy, and at least one tested real-service cancellation flow [1].

Do not overbuild

The brief does not require live bank integration, a machine-learning model, direct payment execution, or automatic revocation. A working prototype using synthetic inputs and a well-tested rules/reference engine is aligned with the supplied brief.

3. Existing products and what they already do

The products below are competitors, adjacent products or platform-level substitutes. Feature descriptions reflect their public product pages, official help pages or app-store listings reviewed on 30 October 2026. Marketing claims are not independent performance audits. “Not publicly documented” does not prove that a feature does not exist.

A. India-focused products

Product	Publicly described capabilities	Implication for our idea
Fold [2]	Detects recurring payments such as subscriptions, rent, EMIs and utilities from transactions; presents a recurring calendar with paid, upcoming and overdue states and reminders.	Recurring-payment discovery and calendar views are already served. Do not differentiate on “one list of subscriptions” alone.
Paylert [3]	Google Play listing claims local SMS-based detection of subscriptions, UPI AutoPay and bank mandates; renewal alerts; trial tracking; unused/duplicate subscription insights and an AI assistant.	SMS detection, AutoPay-related discovery, trial reminders and “unused” insights already appear in a direct competitor’s claims. Focus on transparent evidence, accuracy and cancellation completion.
Fidaura [4]	Google Play listing and product site describe recurring-charge and price-change detection, renewal reminders, CSV/Excel/text-PDF statement import, and on-device encrypted storage.	Statement import and price-rise detection are not unique. The differentiator must be what happens after detection: merchant/rail resolution and guided cancellation.
Reccurra [5]	A browser-based personal finance tracker; users can forward bank alert emails; known formats are parsed, unknown formats may use AI; supports subscription schedules, yearly totals, trial flags, price-rise detection and renewal reminders.	Email ingestion, AI fallback and price-rise/trial functionality already exist in a similar product. Our India-specific data model and cancellation workflow must be sharper.

B. Other India-focused subscription trackers

Product	Publicly described capabilities	Implication for our idea
Snip [6]	Manual subscription entry; renewal reminders; monthly/annual totals; payment-method tagging; category trends; step-by-step cancellation guides for supported services. Its site says it does not connect to banks.	Cancellation guides and annual totals already exist. Potential distinction: automatic discovery plus evidence that links a detected debit to the correct service and payment rail.
Essara [7]	Manual entry or bill scanning; recurring subscription list and renewal reminders; annualised subscription cost and a UPI AutoPay tracker. Its site says it does not request bank login, SMS inbox or UPI PIN.	Manual tracking and UPI AutoPay awareness are already available. Emphasise low-friction ingestion and commitment resolution, while retaining explicit user control.

Competitive conclusion: a feature checklist alone will not make DebitLens distinctive. The product needs a coherent workflow that links transaction evidence, merchant identity, recurrence reasoning and the right cancellation route.

4. Payment rails, platform tools and international references

A. Payment-platform tools in India

Product / initiative	What its official material says	Design implication
Google Pay AutoPay [8]	Users can find live, pending and completed mandates, pause or cancel a live mandate and inspect its frequency/details. Revocation requires UPI PIN.	Do not recreate mandate controls. Link users to the correct app and provide clear instructions; do not ask for or handle a UPI PIN.
PhonePe AutoPay [9]	Terms describe mandate pause, resume, withdrawal/revocation and migration, subject to applicable rules and eligible-merchant conditions.	Cancellation rights and availability can vary by mandate type/merchant. The playbook must include eligibility caveats, not one generic step list.
NPCI UPI AutoPay enhancement [10]	NPCI circular dated 7 October 2025 says users should be able to view active UPI AutoPay mandates in an app of their choice and port mandates between UPI apps. It also restricts use of mandate details obtained through "view mandate" for other purposes.	Do not claim UPI mandates remain permanently scattered across apps. Treat UPI mandate access and permitted reuse as a regulatory/product constraint.
NPCI UPI HELP pilot [11]	NPCI circular dated 8 October 2025 describes a pilot assistant for UPI queries, grievances and mandate lifecycle management, including viewing active mandates and deep links to UPI apps for pause/resume/revoke.	A national-level support/mandate assistant is an adjacent initiative. DebitLens must add value beyond simply surfacing UPI mandates.
RBI e-mandate framework 2026 [12]	The April 21, 2026 framework applies to recurring card, PPI and UPI transactions and covers registration, revocation, notifications and other controls.	Build with compliance awareness. Treat actual mandate operations as the responsibility of the authorised issuer/payment provider, not an unregulated prototype.

B. International reference

Rocket Money [13] is a useful reference for subscription management: its help documentation describes a cancellation assistant for eligible members, with options to request cancellation through its team or view manual instructions. This shows that assisted cancellation is a proven product pattern internationally. It does not establish that the same service model is available or legally/operationally transferable to India.

C. Why platform-level visibility is not the whole answer

A mandate is an authorisation to collect money. A subscription or service relationship is the customer's agreement with a merchant. They are related but not identical. Google Play, for example, explicitly says uninstalling an app does not cancel its subscription [14]. Google Pay and PhonePe explain how to manage UPI mandates [8–9]. The product must teach users to verify both layers when the payment setup requires it.

Competitive reality check

The market is not empty. The opportunity is not "subscription tracker + reminders." The stronger hypothesis is an evidence-backed cross-source commitment record that can explain ambiguous transactions and coordinate the correct service-level and payment-authorisation cancellation steps.

5. Gaps we can try to fill

These are product hypotheses derived by comparing the public capabilities above with HackFront's requirements. We have not proven that no competitor offers them. We should say "a gap in the products and documentation we reviewed," not "nobody has built this."

Opportunity / gap hypothesis	What DebitLens would do	Why it may matter	Validation needed
1. Payment-to-service identity resolution	Map raw descriptor → processor or aggregator → likely merchant → service/subscription, with confidence and evidence.	A person may see a processor or abbreviated string rather than the brand they recognise.	Do users frequently fail to identify these descriptors? Can our curated alias set resolve a useful share correctly?
2. Irregular commitment reasoning	Detect monthly, quarterly, annual, variable-amount, resumed and loosely periodic payments.	Annual and resumed charges are easy to miss in monthly-only logic; the brief singles these out.	Can a rules engine distinguish a recurring commitment from similar one-off purchases without too many false positives?
3. Two-layer cancellation playbook	Separate merchant/service cancellation from payment-mandate or card/e-mandate revocation where applicable.	Stopping the payment authorisation may not terminate the service, and stopping the service may not always revoke a separate mandate.	Can we map each supported mechanism to official, current instructions and test one flow end to end?
4. Post-cancellation evidence trail	Record date, confirmation/reference, service status, mandate status and any later matching debit.	A user needs to know whether the cancellation actually took effect and what evidence to use in a dispute.	Can we define reliable statuses without pretending to verify backend actions we cannot see?
5. Explainable detection and uncertainty	Show which transactions support a recurring pattern, the detected interval, resolver rule and confidence.	A false merchant match or incorrect cancellation path can harm trust.	Will users understand and trust explanations? What confidence threshold should trigger manual review?

Recommended differentiation hierarchy

- Primary: commitment identity + cancellation workflow, backed by evidence.
- Secondary: irregular recurrence and price-change intelligence.
- Supporting: annual commitment view, reminders, unused-subscription prompts and dashboards.

A point to be careful about

"Unused subscription detection" cannot be reliably inferred from bank transactions alone. For the prototype, ask the user for a last-used date or a self-reported usage signal and label the result "potentially unused" rather than asserting it as fact.

6. Main idea: DebitLens

Product definition

DebitLens is a consumer-facing recurring-commitment intelligence tool. It imports transaction records supplied by the user (synthetic data in the hackathon prototype), identifies possible recurring commitments, explains the likely merchant and payment mechanism, detects material changes and guides the user through the appropriate cancellation steps.

Target user for the first version

Initial persona

An urban Indian consumer who pays for several digital services, utilities, memberships or insurance products through a mix of UPI AutoPay, cards and bank-based recurring instructions. They have several months of transaction history but cannot confidently identify every recurring debit or remember how each one should be stopped.

The four user questions the product answers

User question	DebitLens response
“What is taking my money?”	A recurring commitment list with merchant/service name, amount, interval, next expected debit and the transactions behind the match.
“Is this amount changing?”	A comparison against prior payments; show old amount, new amount, percentage/rupee change and annualised impact.
“What is this merchant descriptor?”	A likely merchant match, resolver evidence, confidence level and a request for user confirmation when uncertain.
“How do I stop it?”	A tailored playbook for the underlying service and the relevant payment mechanism, with official links and completion evidence fields.

Proposed product promise

“Understand every recurring commitment we can identify from the records you provide, see why we identified it, and follow a clear route to stop it.”

Avoid saying “we find every auto-debit” until broad coverage has been demonstrated. Use “identified from the records provided” and display missing-data limitations.

7. How DebitLens works: user flow

The initial prototype should be an interactive web application using generated CSV/JSON statement and notification data. The user must be able to inspect and correct imported records rather than being forced to trust an opaque classifier.

1. Add data	User uploads a synthetic CSV or loads a prepared demo dataset. A later version may support consented bank-data sharing or notification/email imports, subject to product, platform and regulatory constraints.
2. Normalise records	Convert each row into a consistent structure: date, amount, debit/credit, raw descriptor, source, account alias, transaction reference and optional notification text.
3. Resolve merchant	Clean the descriptor and compare it with a curated alias/processor/merchant reference set. Return a likely merchant, evidence and confidence; route ambiguous cases to user review.
4. Detect patterns	Group transactions by likely service/merchant and detect periodicity, interval tolerance, amount changes, skipped periods, resumed activity and trial-to-paid events where trial evidence is available.
5. Create commitment record	Link the supporting transactions, merchant, likely service, payment rail, recurrence pattern, projected annual cost and current confidence.
6. Explain alerts	Generate price-rise, annual-renewal, trial-conversion, resumed-payment and potential-unused alerts with plain-language reasoning.
7. Guide cancellation	Show distinct service cancellation and payment-authorisation steps. Use official merchant/payment-provider instructions; the application does not enter credentials or execute the cancellation.
8. Track evidence	Let the user record confirmation number, date, screenshots/reference note and whether service and mandate statuses were confirmed. Flag later matching debits for review.

What the user sees at the end

- A commitment register with “confirmed”, “likely” and “needs review” labels.
- A projected annual commitment total with assumptions clearly shown.
- An alert feed explaining why a payment has been flagged.
- A cancellation checklist and evidence timeline for each selected commitment.

8. Proposed working architecture

Recommended for the hackathon: React + Vite frontend, a small Python FastAPI backend for parsing/detection, and SQLite or JSON storage. If the team is significantly faster in Java, Spring Boot can replace FastAPI; do not introduce two backend stacks just for the diagram.

INPUT LAYER
Synthetic CSV statements • mock SMS/email notifications • prepared test fixtures

↓

NORMALISATION
Date/amount parsing • debit-credit classification • descriptor cleanup • duplicate handling

↓

MERCHANT RESOLUTION
Alias dictionary • payment processor mappings • merchant catalogue • confidence / review status

↓

INTELLIGENCE ENGINE
Recurrence detection • irregular interval analysis • price-change detection • trial events • resumed payments

↓

COMMITMENT GRAPH / DATA MODEL
Transaction -> merchant -> service -> payment rail/mandate -> cancellation playbook

↓

ACTION GUIDANCE
Mechanism-specific steps • official links • service/mandate checklists • evidence timeline

↓

USER INTERFACE
Dashboard • commitment register • alerts • merchant detail • cancellation assistant

Suggested data objects

Entity	Important fields
Transaction	id, date, amount, currency, direction, raw_descriptor, source_type, reference_id
Merchant	merchant_id, canonical_name, aliases, processor_aliases, category, official_url
Commitment	commitment_id, merchant_id, service_name, interval, expected_amount, confidence, status
Evidence link	commitment_id, transaction_id, evidence_type, explanation, confidence contribution
Payment instruction	commitment_id, rail, mandate/card reference alias, provider, status, known limits
Cancellation playbook	service steps, payment-authorisation steps, official source URL, checked date, caveats
Cancellation event	started_at, service_cancelled_at, mandate_revoked_at, confirmation_reference, evidence_note

For the hackathon, this can be implemented with simple relational tables or JSON objects. "Commitment Graph" describes the relationships in the product; a graph database is not mandatory.

9. Detection logic and examples

A. Recurrence detection

Start with explainable rules. Group candidate transactions using merchant/service identity, then compare the time gaps between consecutive transactions. Support weekly, monthly, quarterly, annual and variable/irregular patterns. Use a tolerance window rather than requiring the same calendar day or exact same amount.

Pattern	Illustrative evidence	Expected output
Monthly	₹199 on 5 Jan, 5 Feb, 6 Mar	Likely monthly; 3 supporting transactions.
Annual	₹1,499 on 15 Oct 2024 and 15 Oct 2025	Possible annual commitment; confidence should be lower with only two observations.
Price increase	₹199, ₹199, then ₹249 for same resolved service	Price increase of ₹50 / 25%; show the comparison period.
Resumed payment	Monthly pattern, two skipped cycles, then same merchant debits again	Resumed or irregular recurring activity; review before treating as active commitment.
Trial conversion	Synthetic trial-ending email/notification followed by first paid debit	Trial converted to paid, supported by trial event + debit.

B. Merchant resolver

- Normalise case, punctuation, repeated spaces and common bank abbreviations.
- Maintain explicit alias mappings for known descriptors, processors and merchants.
- Keep processor identity separate from the likely underlying merchant; do not assume a payment aggregator is the service provider.
- Return a confidence level and explanation. For low confidence, ask the user to choose or leave it unresolved.

C. Annual cost calculation

For a fixed known schedule, calculate a projected annual cost from the expected charge and recurrence. For a monthly charge, use 12 × amount; quarterly, 4 × amount; annual, 1 × amount. For variable or uncertain commitments, show an estimated range or “insufficient history” rather than false precision. Do not mix one-time purchases into the recurring total.

D. Metrics for the judges

Metric	How to calculate / present
Recurring-payment recall	True recurring commitments correctly detected ÷ all labelled recurring commitments in the synthetic test set.
Precision	Detected commitments that are actually recurring ÷ all commitments the system flagged.
Merchant match accuracy	Correct merchant matches ÷ merchant descriptors with a ground-truth label; also report unresolved coverage.
Price-change correctness	Correctly identified direction and amount of change across labelled cases.
Cancellation playbook correctness	Human-reviewed checklist matches the relevant service/payment mechanism and official source.

Evaluation discipline: publish the dataset size, labelled cases, true positives, false positives, false negatives and unresolved cases. Do not present a small synthetic test as real-world performance or claim 100% coverage.

10. Cancellation assistant: the product's most important workflow

The cancellation assistant must not treat every recurring payment as a UPI mandate. A recurring commitment may be associated with UPI AutoPay, a card e-mandate, NACH/eNACH, a platform-managed subscription (such as an app store) or a merchant-specific billing arrangement. The available route depends on the actual payment setup.

Special caution: insurance premiums, loan instalments, EMIs and essential utilities must not be handled like optional entertainment subscriptions. Revoking a debit does not necessarily end the underlying policy, loan or service obligation and may create coverage, penalty or repayment consequences. The playbook should direct users to the provider and explain the need to confirm the contract-level effect before acting.

Use a two-layer checklist

Layer	What the user must do	What DebitLens should show
A. End the service relationship	Cancel the subscription, membership, policy or service with the merchant/platform according to its own terms.	Merchant name, official cancellation page/app route, steps, renewal/cut-off caveat and confirmation field.
B. Stop the payment authorisation (where applicable)	Manage/revoke the mandate or card/bank instruction through the relevant authorised provider, subject to its rules.	Payment rail, relevant app/bank/provider, official steps, required authentication caveat and status verification.

Illustrative UPI AutoPay playbook

1. Open the UPI application or bank interface where the mandate is available.
2. Find the relevant live mandate and verify the merchant, frequency, amount and reference.
3. Choose the available pause/cancel/revoke action and complete the provider-required authentication yourself.
4. Check the updated mandate status in the provider application.
5. Separately check whether the underlying merchant subscription/service has been cancelled.
6. Save the service confirmation and mandate-status evidence in DebitLens.

Google Pay's official documentation gives its own AutoPay navigation and says users can verify the resulting status. PhonePe's terms qualify mandate actions by merchant eligibility and applicable rules [8–9]. The above is a conceptual checklist: the final steps must be verified for the exact provider and use case before showing them to users.

Escalation when cancellation is not acknowledged

- Show the merchant's official support and grievance route, and the payment provider/bank dispute route where appropriate.
- Tell the user what evidence to retain: transaction date, amount, merchant descriptor, mandate reference and merchant cancellation request/response.
- Do not promise a refund, successful cancellation or guaranteed stop to future debits.
- If a later debit appears after the recorded cancellation date, label it "possible post-cancellation charge — review" until the user verifies it.

Prototype boundary

DebitLens should explain, link and track. It should not collect UPI PINs, bank passwords, card credentials or OTPs; it should not impersonate the bank/UPI app; and it should not attempt to revoke a real mandate from the hackathon prototype.

11. What we should actually build for the hackathon

Must-have MVP

Priority	Feature	Definition of done
P0	Synthetic data import	One CSV/demo dataset loads reliably; parser handles date, amount, descriptor and transaction type.
P0	Recurring commitment detection	Demonstrates at least monthly plus annual/irregular/resumed cases with supporting transactions.
P0	Merchant resolver	Maps a curated set of descriptors to canonical merchants and shows confidence/evidence.
P0	Price-change alert	Correctly identifies at least one synthetic amount increase and calculates its annual impact.
P0	Annual view	Shows monthly and annual commitment totals with transparent schedule assumptions.
P0	Cancellation playbook	Provides one thoroughly checked real-service cancellation example and at least two additional illustrative rail-specific templates.
P1	Trial conversion alert	Uses a synthetic trial event and matching first debit.
P1	Cancellation evidence timeline	Allows a user to record confirmation date/reference and track service/mandate statuses separately.
P2	Usage-based “potentially unused” alert	Uses a user-confirmed last-used date; do not infer app use from bank transactions.

Explicitly out of scope for the first prototype

- Live bank/Account Aggregator integration, production SMS scraping or broad email access.
- Automatic UPI mandate revocation, payment initiation or collection of PIN/OTP/password information.
- A custom large language model, sophisticated ML forecasting or a complex graph database.
- A claim that all merchants, banks, cards or mandate types are supported.

Suggested demo storyline

1. Load a 12–18 month synthetic statement containing monthly, quarterly, annual, variable and resumed payments.
2. Show the system detecting a group of commitments and explaining the transactions behind each one.
3. Open an ambiguous descriptor and show the processor/merchant resolution with a confidence explanation.
4. Show a price increase and an annual renewal that would be missed by a monthly-only view.
5. Open the cancellation assistant, show the two-layer checklist and official source link, then record a test cancellation confirmation.
6. Finish with annual spend, potential savings and the measured test-set results.

12. Potential of the solution and what could come next

User value

- Visibility: one view of recurring commitments detected from records the user supplies.
- Comprehension: plain-language explanation of confusing descriptors and detection evidence.
- Control: a verified route to the merchant and payment provider, rather than a generic “cancel” button.
- Financial awareness: annualised cost and price changes make small recurring charges easier to evaluate.

Possible product evolution

Stage	Possible development	What must be validated first
Prototype	Synthetic CSV + curated merchant reference + rules-based recurrence + cancellation playbooks.	Detection quality, explainability, usefulness and task completion with test users.
Pilot	User-controlled statement import and notification/email parsing, with explicit consent and deletion controls.	Device/platform constraints, privacy controls, support burden and user trust.
Partnership	Explore integrations or distribution through banks, issuers, regulated Account Aggregator ecosystem participants or consumer-finance platforms.	Eligibility, permissions, business agreements, regulatory role and value to partner.
Scale	Broader merchant catalogue, more payment rails, regional-language guidance, family/shared commitments and reliable post-cancellation monitoring.	Accuracy across more banks/providers, service updates, localisation and ongoing maintenance.

Potential business models — hypotheses, not facts

- B2C freemium: free core tracker; paid tier for more history, advanced insights, household view or richer cancellation tracking.
- B2B2C distribution: a bank, card issuer or consumer-finance app could offer recurring-commitment visibility as a trust/retention feature.
- Partner referrals: only carefully disclosed, non-conflicting referrals; never allow commercial incentives to bias merchant identification or cancellation advice.

Who pays and why?

The initial hypothesis is that consumers pay for convenience and better visibility, but willingness to pay may be low because a subscription-management tool is itself competing for the user’s money. A partner-funded model may be more viable if a bank or financial platform sees measurable customer value. The team should test rather than assume this.

A useful three-month direction

Move from a synthetic-data prototype to a consent-based pilot, expand the merchant-reference library, measure detection across varied statement formats, maintain an audited cancellation knowledge base, and validate one distribution or partnership path.

13. Risks, limitations and trust principles

Risk	Why it matters	Mitigation in the prototype
False positive recurrence	A repeated merchant name may represent unrelated purchases, not a subscription.	Require multiple observations or stronger evidence; show confidence and allow dismissal/correction.
Wrong merchant resolution	A processor/aggregator descriptor may not identify the actual service.	Separate processor from merchant; expose alias evidence; leave ambiguous rows unresolved.
Wrong cancellation guidance	Different rails, merchants, plans and jurisdictions may have different routes.	Use official sources, display checked date and caveats, test the exact use case and avoid universal claims.
Service and mandate confusion	Revoking payment authority may not end the service contract; cancelling service may not always remove a separate payment instruction.	Separate the two actions and statuses in the interface.
Usage cannot be inferred	Payment history does not prove whether a service was used.	Ask user to provide last-used date or opt-in usage data; label only as “potentially unused”.
Sensitive financial information	Statements and notifications can contain personal identifiers and account details.	Use synthetic data in the hackathon; minimise fields; redact samples; avoid storing credentials; define deletion.
Regulatory / platform limits	UPI and e-mandate information is governed by NPCI/RBI and provider rules; AA data sharing is consent-based and role-restricted.	Do not scrape or repurpose mandate data without authority; explore compliant integration only with appropriate partners.
Stale playbooks	Merchants change settings menus and cancellation policies.	Store source URL, verification date, owner and re-check schedule for each playbook.

Trust principles we should adopt

- Evidence over confident-sounding AI: every detected commitment should link to supporting records or a rule.
- User control: the user reviews merchant matches and decides whether to follow cancellation steps.
- No credential collection: no PINs, OTPs, bank passwords or card security codes.
- Honest status: distinguish “instructions opened”, “user says cancelled”, “confirmation uploaded” and “provider status verified”.
- Privacy by design: collect the minimum data needed, explain what is processed, provide deletion/export and avoid using real financial data in the hackathon demo.

14. Team feedback and decision questions

Use these questions in the team meeting. The purpose is to challenge the idea before committing to implementation, not to defend the current concept at all costs.

Problem and user

- Who is the first user we are designing for: students, working professionals, families, or another clearly described group?
- What is the one painful situation we want to solve first: unknown merchant descriptors, missed annual renewals, price increases, or cancellation confusion?
- What evidence do we have that our target users experience this problem? Can we speak to 3–5 people tonight?

Competitive positioning

- Which existing product is our closest competitor: Paylert, Fidaura, Fold, Snip, Reccurra or a payment app?
- Which capability do we believe is genuinely better in our prototype, and what evidence will demonstrate that?
- Are we adding value beyond what NPCI UPI mandate visibility and UPI HELP initiatives already provide?

Data and detection

- What is our first supported input: CSV only, CSV plus mock notifications, or another manageable combination?
- Which recurrence patterns will we support reliably by the deadline?
- How will we handle false positives, variable amounts, refunds, split purchases and changing merchant descriptors?
- What confidence threshold triggers “needs review” rather than an automatic merchant match?

Cancellation and trust

- Which exact real service can we test for the required end-to-end cancellation demonstration?
- How will we show service cancellation separately from mandate/card/bank-authorisation revocation?
- Who in the team verifies the official cancellation guide and its last-checked date?
- What must the app explicitly refuse to do?

Business and future

- Would an individual consumer pay for this? What feature would justify payment?
- Could a bank/issuer/consumer-finance partner benefit, and why would it distribute the product?
- What is the biggest obstacle to turning the demo into a real product: data access, accuracy, merchant knowledge, regulation, or distribution?

Build decisions

- What is the single end-to-end flow we can finish and test?
- Who owns frontend, data/detection, cancellation knowledge base and integration?
- Which features are we deliberately cutting to protect the working prototype?
- What result would make us change the idea rather than continue building it?

15. Decisions to record before coding

Complete this as a team and keep it as the agreed scope for the sprint.

Decision	Team answer / owner
Chosen product name (working name: DebitLens)	
Primary user and core pain point	
One-sentence product promise	
Supported input format(s)	
Supported payment mechanisms in the demo	
Merchant reference set owner	
Detection patterns included	
Exact real-service cancellation test	
Frontend / backend / deployment stack	
Owner for each module	
Measurable demo success criteria	
Features explicitly out of scope	

Suggested sprint order (deadline: tomorrow night)

Order	Workstream	Deliverable
1	Agree scope + create fixtures	A labelled synthetic dataset, expected outcomes and merchant alias map.
2	Build the core data path	CSV import → normalised transactions → recurring commitment list.
3	Add intelligence	Annual/irregular detection, one price-rise case, merchant confidence and annual spend.
4	Build cancellation experience	One validated real-service flow, two-layer checklist, official links and evidence fields.
5	Integrate and test	Run all test cases; record precision/recall and merchant-match results; fix critical failures.
6	Deploy and present	Live working demo, backup recording, concise explanation of the gap and limitations.

Final go/no-go test

Before spending time on polish, the team should be able to complete one coherent journey from imported synthetic transaction to identified commitment to evidence-backed cancellation guidance. If that journey does not work, cut secondary features and finish it first.

16. Research references and source notes

Research snapshot prepared on 30 October 2026. Product descriptions are based on public product pages, app listings or official help/terms pages available at the time of review. App functionality, policies and availability may change. This is product research, not legal advice or a claim of exhaustive market coverage.

Hackathon source

[1] HackFront India, Edition 01 Problem Statements, Theme T4 Fintech, PS T4.1 "The Auto Debits Nobody Can Find", pp. 11–12. Source supplied by the team; use the original PDF for the exact wording.

India-focused products

[2] Fold FAQ — recurring payments and subscriptions. Open source — Fold says it detects recurring payments from transaction data and shows a paid/upcoming/overdue calendar.

[3] Paylert — Google Play listing. Open source — The listing describes SMS-based recurring subscription, UPI AutoPay and bank-mandate detection, trial reminders and insights.

[4] Fidaura product site and Google Play listing. Open source — The site/listing describe recurring-charge and price-change detection, statement import and on-device storage.

[5] Reccurra — website, features and help. Open source — The site describes bank-alert email forwarding, subscription tracking, recurring schedules, trial flags and price-rise detection.

[6] Snip — Subscription Tracker. Open source — The site describes manual subscription tracking, annual totals, renewal reminders, payment-method tags and cancellation guides.

[7] Essara — Expense, Subscription & UPI AutoPay Tracker. Open source — The site describes subscription tracking, annualised totals, bill scanning and its privacy boundaries.

Payment rails, NPCI and regulation

[8] Google Pay Help — Learn how to manage recurring payments / AutoPay. Open source — Official instructions for finding, pausing, revoking and verifying UPI AutoPay mandates in Google Pay.

[9] PhonePe — AutoPay Terms and Conditions. Open source — Official terms describe available mandate actions and conditions/limitations.

[10] NPCI Circular UPI/OC-223/2025-26, 7 October 2025 — Enhancement of UPI AutoPay. Open source — Describes user-driven mandate view/porting across UPI apps and restrictions on use of mandate data.

[11] NPCI Circular UPI/OC-227/2025-26, 8 October 2025 — UPI HELP pilot. Open source — Describes UPI HELP support for queries, grievances and mandate lifecycle management with deep links.

[12] RBI Digital Payments — E-mandate Framework, 2026, dated 21 April 2026. Open source — Framework for recurring card, PPI and UPI transactions; verify official current text before production use.

International reference, platform cancellation and data sharing

[13] Rocket Money Help Center — Subscription Cancellation Assistant. Open source — Describes manual cancellation instructions or cancellation requests handled by Rocket Money for eligible members.

[14] Google Play Help — Cancel, pause or change a subscription. Open source — Official guide states uninstalling the app does not cancel its subscription.

[15] Government of India, Department of Financial Services — Account Aggregator Framework. Open source — Explains the consent-based financial-data-sharing framework and roles of AAs/FIPs/FIUs.

How to read this research

- Verified public capability: the capability is stated in a cited official product page, help page, terms page or app listing.
- Opportunity hypothesis: a proposed differentiation inferred from comparison; it must be tested with users and a broader market review.
- Not established: a product page does not mention a feature. This is not proof that the product lacks it.
- Regulatory caution: the prototype uses synthetic data. Any real integration requires separate product, legal, security and partner review.